

3D 단지뷰 성능 개선 현황과 추가 개선 분석

gpt-6.1-Sol 작업(2026-10-02~03) 전수 검토 · 로딩/지연 잔여 병목 · 우선순위별 개선안 | 2026-10-05 · 작성: Claude Code

1. 요약

결론. Sol의 작업은 **모델 구축(1112→263ms, 4.2배)**, CDN 장애 내성(11.4s→2.0s), 숨은 뷰 GPU 정지, 자원 수명 관리 측면에서 실질적인 개선이었다. 그러나 사용자 수용 기준(첫 표시 3배 단축, 로딩 중 입력·프레임 지연 ≤20ms)은 **Sol 스스로의 판정 스크립트(check-complex-targets.py)에서도 실패**했다. 첫 표시는 1862→1410ms(1.32배)에 그쳤고, 로딩 중 최대 프레임 간격 125~132ms, 최대 입력 지연 99~128ms가 남아 있다. WebGL 경로는 로딩 중 852ms 정지가 있다.

- 남은 지연의 대부분은 **모델 구축 이후** 구간에 있다. 1410ms 중 모델 구축은 263ms뿐이고, 나머지 약 1.1초는 데이터 대기·페인트/텍스처 준비·파이프라인 컴파일·GPU 업로드·첫 프레임 제출이다.
- 로딩 중 120ms대 긴 프레임의 원인 후보는 (1) 동기 파이프라인 컴파일 대체 경로(GPU.ready → createRenderPipeline), (2) 큰 버퍼·텍스처의 한 프레임 집중 업로드, (3) 표시 직후 메인 스레드 생성 작업(보행자·식재·교통·사진 분석)이다. 셋 다 계측 지점이 이미 코드에 있어 바로 확인할 수 있다.
- 정상 상태에서도 렌더 루프 자체가 1배속 기준 프레임당 약 6ms(드로우 705회)를 메인 스레드에서 쓴다. CPU가 4배 느린 기기에서는 이것만으로 약 24ms가 되어, 로딩과 무관하게 40fps 상한이 생긴다. 근본 해결책은 렌더러를 워커(OffscreenCanvas)로 옮기는 것이다.
- Sol이 만들었지만 **연결하지 않은 코드**가 작업 트리에 남아 있다: ShaderText/ShaderTextWorker(셰이더 전처리 워커), trafficCollision.ts(할당 없는 SAT 충돌), test-complex-frame-budget.mjs. 어디에서도 import되지 않는다.
- 권장 순서: **P0** 긴 프레임 3종 제거(1~2주, 로딩 중 최대 프레임 간격 ≤33ms 목표) → **P1** 사전 베이킹 장면 팩과 점진 표시(첫 표시 ≤700ms 목표) → **P2** 렌더러 워커 분리(입력 지연 약 0, 저사양 60fps).

2. 검토 범위와 근거

구분	대상	비고
Sol 커밋	647f48e (10-03) Improve 3D texture loading and resource lifecycle — 77개 파일, ComplexHologram +1140줄	사전 생성 페인트 35종, 워커 래스터, 자원 소유권, 운전 모듈 일괄 포함
Sol 커밋	1702e5b (10-03) Fix 3D loading stalls and add explicit performance acceptance checks — 19개 파일	회귀 복구, CDN 경쟁 요청, 분할 업로드, 공유 텍스처 풀, 판정 스크립트
Sol 커밋	a2e9585 배포 기록	수치 판정 실패 상태로 사용자 요청에 따라 배포
Sol 문서	docs/realestate-3d-performance-2026-10-02.md, realestate-3d-recovery-2026-10-03.md, realestate-3d-deployment-2026-10-03.md	측정 원본은 tmp/*.json
미연결 WIP	vendor/tidewater/engine/gpu/ShaderText.js · ShaderTextWorker.js · ShaderAnalysis.js, components/trafficCollision.ts, scripts/test-complex-frame-budget.mjs	untracked, import 없음
비교 기준	5215696 (10-02, Sol 이전 마지막 배포) 및 10-01/10-02 작업 기록	4배 감속 재현 결과 포함

이번 보고서는 코드와 Sol이 남긴 측정 원본을 읽어 분석했다. 새로운 브라우저 측정은 하지 않았다. 아래의 기대 효과 수치는 기존 측정과 코드 구조에서 추정한 값이며, 각 항목에 검증 방법을 함께 적었다.

3. Sol이 개선한 부분

3.1 로딩 경로

영역	변경 내용	효과 · 근거
외벽 페인트	원본 페인터를 워커(OffscreenCanvas)로 이동하고, 단지와 무관한 주변 건물 맵·노멀 35종(3.77MB PNG)을 사전 생성했다(paintAssetsPlugin이 소스 해시를 검사). 실패하면 원본 페인터로 대체한다.	모델 구축 1112→263ms(4.2배). 픽셀 차이 최대 1/255
캐시 저장	전경에서 하던 createImageBitmap(texture.image) 복사(약 475ms)를 없앴다. 페이지가 숨겨졌을 때 워커가 PNG 캐시를 만든다.	로딩 중 큰 정지 1건 제거
CDN 대기	CDN과 같은 원본(origin) 요청을 250ms 간격으로 경쟁시키고 진 요청은 취소한다. 요청마다 10초 제한, 사전 페인트는 3초 실패 시 페인터로 전환한다.	CDN 5초 지연 조건에서 11402→2005ms
지형·교량·보도	지형 정점·법선·UV·인덱스, 교량 탐색, 보도 경로를 같은 알고리즘 그대로 sceneWorker로 옮겼다(배열 전송).	320분할 지형 결과 일치 검증
GPU 업로드	큰 비트맵을 행 단위로 나눠 올리고, 고해상도 재질 교체에도 준비 검사를 적용했다. 채널 패킹도 분할했다.	픽셀 차이 0, 단일 프레임 정지 완화
WebGPU 선택	선택적 WGSL 기능 표식 대신 실제 어댑터로 시도한다. 실패한 파이프라인은 watchdog를 기다리지 않고 WebGL로 전환한다.	불필요한 WebGL 낙하 방지
파이프라인	쓰지 않는 셰이더 함수를 제거하고 같은 프로그램의 파이프라인을 공유한다. 미리 시작한 맵·패킹 파이프라인을 기다린다.	동기 재컴파일 경로 감소

3.2 렌더링 · 자원 수명

영역	변경 내용	효과 · 근거
숨은 뷰	숨김·화면 밖·일시정지 뷰의 렌더 루프를 멈추고 HDR/AO/TAA/bloom 임시 타깃을 반환한다(suspendTargets).	숨은 뷰 렌더 호출 0회. 조망 구간 실제 렌더 90→180회/180프레임
공유 텍스처	같은 고정 페인트·데이터 맵을 장치별 참조 카운트 풀로 공유한다(sharedTextures.js).	두 뷰 407.5→365.7MB(-10.3%)
소유권	SceneResources/sceneMemory로 취소·오류·교체 때 지오메트리·재질·비트맵·워커를 회수한다. 늦게 도착한 결과는 epoch로 폐기한다.	8회 열기/닫기 후 텍스처 69개로 일정, 단으면 0.7MB
캐시 상한	셰이더 캐시 LRU, IndexedDB 건물 60·페인트 40개, 래스터 워커 최대 2개, 5초 유휴 시 종료	장시간 사용 누수 방지
품질 원칙	동적 해상도·품질 하향과 30fps 유휴 제한을 제거하고, GPU 여유가 있으면 슈퍼샘플링을 유지한다.	'고품질 대원칙'에 부합

3.3 검증 체계

- test-complex-performance.py(--production, --cpu-rate 4, --webgl, --fail-webgpu-pipeline, --paused-rail), check-complex-targets.py, verify-complex-performance.py, test-complex-leak/paint/upload/gpu/memory를 추가했다. **판정 스크립트가 종료 코드로 실패를 알린다는 점이 가장 큰 자산이다.**
- 한계: 측정은 합성 단지(본 건물 9, 주변 275) + 고정 API 응답 + 고정 렌더 배율 1.5 조건이다. 실제 도시 데이터(1km 링, 교량, 물, 야간), 실제 휴대폰, 브라우저 전체 메모리는 검증되지 않았다.

4. 현재 수치와 목표의 차이

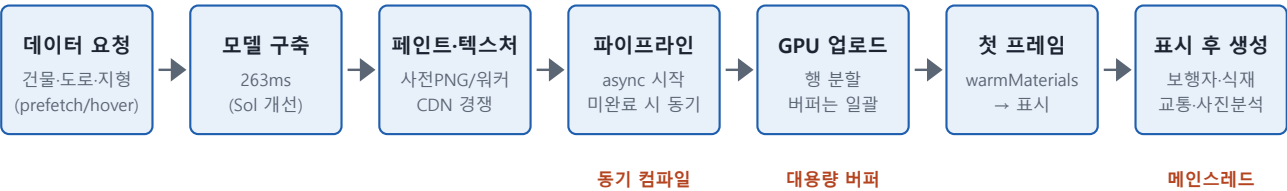
지표 (일반 조건, 3회 중앙값/최대)	5215696 (이전)	1702e5b (Sol)	목표	판정
선택 → 첫 표시	1862ms	1410ms	≤ 620.7ms (3배)	미달 (1.32배)
모델 구축	1112ms	263ms	—	달성 (4.2배)
로딩 중 최대 프레임 간격	300.5ms	125.2ms (최대 132.4)	≤ 20ms	미달
로딩 중 최대 입력 지연	248.2ms	125.3ms (최대 127.9)	≤ 20ms	미달

지표 (일반 조건, 3회 중앙값/최대)	5215696 (이전)	1702e5b (Sol)	목표	판정
표시 후 20ms 초과 프레임	—	3 / 2160 (최대 35.8ms)	0	근접
CPU 4배 감속 첫 표시 / 최대 프레임	—	2596ms / 266.4ms	—	저사양 취약
WebGL 강제 경로 로딩 최대 프레임	—	852.3ms	≤ 33ms	심각
단일 뷰 GPU 텍스처 추정치	234.9MB	234.9MB	큰 폭 감소	변화 없음

해석. 모델 구축을 263ms까지 줄였으므로 남은 1.1초와 120ms대 정지는 "계산" 문제가 아니라 **순서(크리티컬 패스)와 프레임 경계 관리**의 문제다. 더 빠른 계산보다는 무엇을 먼저 보여 주고, 무거운 작업을 어떤 단위로 프레임에 나눠 넣는지가 다음 성과를 결정한다.

5. 잔여 병목 분석

그림 1. 선택 → 표시 → 안정화 경로 (빨간 글씨 = 긴 프레임 후보)



#	병목	코드 근거	증상
B1	동기 파이프라인 컴파일 대체 경로	GPU.ready()가 async 미완료 시 createRenderPipeline 동기 호출(syncCompiles에 기록). texturePack.pipelineFor, Mipmaps도 동기 생성 경로가 있다.	첫 방문/새 재질 조합에서 GPU 프로세스 정지(컴파일 1건당 약 20ms 실측, 여러 건이 겹치면 누적)
B2	버퍼 일괄 업로드	텍스처는 행 분할했지만 병합 지오메트리(주변 275동, 링 600m) 버퍼는 한 번에 writeBuffer/생성	표시 직전 한 프레임에 큰 버퍼 전송이 몰림
B3	표시 후 메인 스레드 생성	afterShown 큐: 보행자 cutPaths/crowd, buildPlants, buildTraffic, parcelEdges, surveyedShape(사진), sharpenNeighbourhood(4초 후)	표시 직후 수 초간 30~130ms 프레임 산발
B4	렌더 루프 CPU 비용	프레임당 약 6ms(1배, 드로우 705). 교통 near-mesh 인스턴스 재패킹, 매 프레임 행렬·유니폼 갱신	4배 기기는 로딩 없이도 약 40fps 상한
B5	WebGL 경로	프로그램 수가 많고 KHR_parallel_shader_compile 미지원 환경에서 동기 링크	로딩 중 852ms 정지
B6	크리티컬 패스 직렬성	첫 프레임이 '완성된 장면'을 기다린다(페인트·파이프라인·업로드 모두 완료 후 표시)	첫 표시 1.4s, 4배 2.6s
B7	미연결 최적화	ShaderTextWorker, trafficCollision.ts가 작성만 됨	기대 효과 미실현

6. 추가 개선 방안 (우선순위)

품질 하향(해상도·효과 축소)은 쓰지 않는다. 모든 항목은 같은 픽셀 결과를 유지하면서 작업의 위치(스레드), 시점, 단위를 바꾸는 방식이다.

ID	개선안	내용	기대 효과(추정)	검증
P0-1	동기 컴파일 0건 보장	첫 표시 전에 GPU.pipelinesReady()를 확실히 기다리고, 표시 후 새 재질이 생기면 그 메시는 파이프라인이 준비될 때까지 그리지 않는다(한 프레임 늦게 보임). texturePack.Mipmaps의 동기 경로를 warm*() 비동기로 통일한다. 지도 페이지 유희 시간(단지 hover 전)에 '공통 파이프라인 세트'를 미리 컴파일한다.	로딩 중 긴 프레임의 최대 원인 제거. 추정 최대 프레임 125ms → 40ms 이하	GPU.syncCompiles 길이가 0 단언을 test-complex-performance에 추가

ID	개선안	내용	기대 효과(추정)	검증
P0-2	업로드 예산 스케줄러	텍스처·버퍼 업로드를 하나의 큐로 모으고 프레임당 바이트·시간 예산(예: 4MB 또는 2ms)을 둔다. 큰 버퍼는 mappedAtCreation과 청크 writeBuffer로 나눈다. device.queue.onSubmittedWorkDone으로 GPU 적체를 감지해 예산을 조절한다.	표시 직전 대용량 전송 프레임 제거. 4배 감속에서 효과가 크다	프레임별 업로드 바이트 HUD(?hud=1) 기록, p99 프레임 간격
P0-3	표시 후 생성 작업 워커 이전	sceneWorker에 crowd/cutPaths, buildPlants 배치 계산, buildTraffic의 경로·교차로 그래프, parcelEdges를 옮긴다. 메인 스레드는 결과 배열로 InstancedMesh를 채우기만 한다. 사진 분석은 이미 워커이므로 결과 반영만 분할한다.	표시 후 산발 정지 제거, 입력 지연 감소	LoAF(Long Animation Frames) API로 스크립트별 귀속 확인
P0-4	미연결 WIP 정리	trafficCollision(할당 없는 SAT-궤적 캐시)을 sceneStreet의 차량 충돌 검사에 연결하고, ShaderTextWorker로 셰이더 전처리·조합을 옮긴다. 각각 원본과 동일 결과를 테스트로 보장한다(테스트 파일은 이미 있음).	GC 정지와 셰이더 문자열 처리 비용 감소	test-complex-frame-budget.mjs를 CI 스크립트에 포함
P1-5	점진 표시(Progressive first frame)	1단계: 지형과 건물 매스(공유 재질, 사전 컴파일된 파이프라인)를 약 300~500ms 안에 보여 준다. 2단계: 외벽 페인트·창문·식재를 0.3초 크로스페이드로 채운다(깜빡임 금지 원칙에 따라 팝 없이 페인트). '완성된 장면 대기'를 끊는 것이 첫 표시 3배의 유일한 현실적 경로다.	첫 표시 1410 → 약 500ms(추정, 3배 기준 충족 가능)	선택→첫 의미있는 프레임 / 선택→완성 프레임을 분리 측정
P1-6	단지별 사전 베이킹 장면 팩	서버 배치(헤드리스)로 단지별 병합 지오메트리(양자화 없는 Float32) + 페인트 맵(무손실 PNG, 또는 이미 허용된 BC7)을 jsDelivr/CDN에 올린다. 첫 방문에서도 '다운로드 → 업로드'만 남는다. 소스 해시가 바뀌면 무효화한다(paintAssetsPlugin 방식 확장).	모델 구축·페인트 시간 0에 근접. 인기 단지 상위 N개부터 적용	팩 유무 A/B, 원본과 픽셀 비교(test-complex-paint 재사용)
P1-7	렌더 루프 CPU 다이어트	정적 메시는 render bundle에 영구 기록(이미 일부), 교통 near-mesh 재패킹은 변경된 슬롯만 갱신, 유니폼은 프레임당 1회 단일 버퍼로 기록, 카메라가 멈춘 정지 장면을 그림자·반사 갱신을 건너뛰는다.	6ms → 3ms 이하(추정). 4배 기기 60fps 근접	Chrome 트레이스 ms/s 비교(같은 페이지 토크)
P1-8	WebGL 경로 보강	KHR_parallel_shader_compile가 없으면 프로그램을 유틸 시간당 하나씩 링크하고, 준비되지 않은 재질은 그리지 않는다. 프로그램 변형 수를 공유 셰이더 기준으로 줄인다.	852ms 정지 → 수십 ms	--webgl 측정 3회
P2-9	렌더러 워커 분리(OffscreenCanvas)	transferControlToOffscreen으로 WebGPU 렌더러와 장면 그래프를 워커로 옮긴다. 메인 스레드는 입력·UI·피킹 요청만 담당한다(피킹은 워커에 질의). Three.js WebGL 대체 경로도 OffscreenCanvas를 지원한다.	로딩 중 입력 지연 약 0. 메인 스레드 정지가 화면 프레임에 전파되지 않음	4배 감속 입력 지연 p99 ≤ 20ms
P2-10	메모리 구조 개선	주변 건물 텍스처를 텍스처 배열/아틀라스로 묶고 거리 기반 밍 스트리밍(먼 링은 상위 밍만 상주)을 적용한다. 해상도 자체는 유지하고, 가까이 가면 원래 해상도가 올라온다.	단일 뷰 234.9MB → 150MB대(추정)	test-complex-memory + 프로세스 RSS
P2-11	운영 계측(RUM)	운영 사용자에게서 첫 표시, LoAF 상위 스크립트, 렌더러 종류(WebGPU/WebGL), 기기 등급을 수집한다(기존 activity 로깅 경로 활용). 합성 단지만으로는 실제 1km 랑·물·야간 장면을 대표할 수 없다.	실제 기기 기반 우선순위 결정	관리자 대시보드 패널

7. 실행 로드맵과 목표치

단계	기간(추정)	포함 항목	통과 기준 (check-complex-targets 확장)
1단계	1~2주	P0-1 ~ P0-4	일반 조건 로딩 최대 프레임 ≤ 33ms, syncCompiles 0, 4배 감속 최대 프레임 ≤ 100ms

단계	기간(추정)	포함 항목	통과 기준 (check-complex-targets 확장)
2단계	2~3주	P1-5 ~ P1-8	첫 의미있는 프레임 $\leq 620\text{ms}$ (3배), 완성 $\leq 1.2\text{s}$, WebGL 최대 프레임 $\leq 50\text{ms}$
3단계	3~4주	P2-9 ~ P2-11	로딩 중 입력 지연 p99 $\leq 20\text{ms}$ (4배 감속 포함), 단일 뷰 GPU 추정 -30%

측정 규칙(기존 메모 반영)

- 비교는 프로덕션 빌드(vite preview) 기준선을 같은 시점에 함께 띄워 A/B $\times 3$ 으로 한다. 개발 서버 수치는 쓰지 않는다.
- rAF 간격과 함께 Chrome 트레이스의 CrGpuMain(셰이더 모듈 생성, 동기 파이프라인, 2D 래스터)을 확인한다. 메인 스레드 프로파일만으로 판단하지 않는다.
- 클릭 흐름(지도 → 카드 3D 버튼)과 단지 이동(hop)을 모두 측정한다. 딥링크 냉시작은 페이지 부트 비용이 섞이므로 따로 본다.
- 기능 통과와 성능 통과를 구분하여 보고한다. 판정 스크립트가 실패하면 '완료'로 보고하지 않는다.

8. 리스크와 주의점

- 점진 표시(P1-5)는 '깜빡임 금지' 원칙과 충돌할 수 있다. 팝 대신 페이드로 전환하고, 카메라 거리에 따라 LOD를 바꾸지 말고 고정 배분해야 한다.
- 사전 베이킹(P1-6)은 단지 수만큼 저장소가 필요하다. 접근 로그 상위 단지부터 적용하고, 나머지는 현재 경로로 대체한다.
- 렌더러 워커 분리(P2-9)는 피킹, 팻말 캔버스, 전체화면 전환, 영상 촬영 혹까지 영향을 준다. 기능 플래그(?worker=1)로 병행 운영한 뒤 전환한다.
- 현재 작업 트리에 3D 외 영역(주식 차트, 후원)의 미커밋 변경과 드라이브 연속 주행(WIP)이 섞여 있다. 성능 작업 전에 브랜치를 분리하기를 권한다.